

Machine Learning

Day 19

LLMs & Prompt Engineering

Beginner Friendly

Theory + Code

6 Hours

Easy

Talking to AI the right way — from zero-shot to your first API call

Contents

01	What are Large Language Models?	3
02	Prompt Engineering Techniques	5
03	Writing Effective Prompts	8
04	Using the Anthropic / OpenAI API	10
05	Lab — Build a Study-Buddy Chatbot	13
06	Quick Reference Cheat Sheet	15

01 What are Large Language Models?

Giant autocomplete engines that learned to read, reason, and write.

A Large Language Model (LLM) is a neural network trained on enormous amounts of text — books, websites, articles, code — to learn the statistical patterns of human language. After training, it can read text and generate new text that continues naturally from it.

The core idea: next-word prediction

At its heart, an LLM does one thing repeatedly: given some text so far, predict the most likely next word (technically, the next 'token'). It does this one token at a time, feeding its own output back in, until it produces a full response. What makes modern LLMs remarkable is scale — billions of parameters trained on trillions of words let this simple mechanism produce reasoning, code, poetry, and conversation.

Concept

```
# Conceptually, an LLM is repeatedly doing this:
prompt = 'The capital of France is'
# Step 1: model sees 'The capital of France is'
# Step 2: predicts next token -> ' Paris'
# Step 3: appends it -> 'The capital of France is Paris'
# Step 4: predicts next token -> '.'
# ...repeats until a stop condition is reached
```

The main players

Model family	Made by	Notable for
GPT-4 / GPT-4o	OpenAI	General-purpose, widely integrated
Claude	Anthropic	Long context, careful reasoning, safety focus
Gemini	Google	Multimodal (text, image, video, audio)
LLaMA	Meta	Open-weight, runs locally
Mistral	Mistral AI	Efficient open-weight models

Key insight

LLMs don't 'know' facts the way a database does — they generate the most statistically likely continuation of text. This is why they can be confidently wrong (a 'hallucination'). Always verify critical facts from another source.

02 Prompt Engineering Techniques

Different ways to structure your request to get better answers.

Prompt engineering is the practice of crafting your input (the 'prompt') so the model produces the most useful possible output. The same model can give a mediocre answer or a brilliant one depending purely on how you ask.

Technique 1 — Zero-shot prompting

Just ask the question directly, with no examples. Works well for tasks the model has seen many times during training.

Example

Prompt:

"Classify the sentiment of this review as positive, negative, or neutral:

'The food was cold and the service was slow.'"

Output: Negative

Technique 2 — Few-shot prompting

Give 2-3 examples of the input/output pattern before asking your real question. This 'teaches' the model your exact format and improves consistency.

Example

Prompt:

"Convert these to formal English:

Informal: 'gonna head out, ttyl'

Formal: 'I am going to leave now. Talk to you later.'

Informal: 'wanna grab lunch?'

Formal: 'Would you like to get lunch together?'

Informal: 'cant make it 2morrow, sry'

Formal: "

Output: 'I will not be able to make it tomorrow. I apologize.'

Technique 3 — Chain-of-thought (CoT)

Ask the model to 'think step by step' before giving the final answer. This is especially powerful for math, logic, and multi-step reasoning problems — it gives the model 'room' to work through the problem instead of jumping to a guess.

Example

Prompt (without CoT):

"A shirt costs \$40. It's on sale for 25% off, then there's an extra 10% off

the sale price. What's the final price?"

-> Models sometimes jump straight to a wrong answer.

Prompt (with CoT):

"...Think step by step before giving the final answer."

Output:

Step 1: 25% off \$40 = \$40 - \$10 = \$30

Step 2: 10% off \$30 = \$30 - \$3 = \$27

Final answer: \$27

Technique 4 — System prompts

A system prompt sets the AI's role, tone, and behaviour for the whole conversation — separate from the user's actual question. It's like giving the model a job description before the conversation starts.

Example

System prompt:

"You are a patient, friendly tutor who explains concepts using simple everyday analogies. Avoid jargon. Keep answers under 100 words."

User: "What is recursion?"

Output:

"Recursion is like a set of Russian nesting dolls. Each doll contains a

smaller version of itself, until you reach the tiniest doll that doesn't

open anymore. In programming, a recursive function calls itself with a smaller version of the problem, until it reaches a 'base case' that doesn't need to call itself again."

Technique	Best for	Trade-off
Zero-shot	Simple, common tasks	May be inconsistent in format
Few-shot	Specific formats/styles	Uses more tokens (cost)
Chain-of-thought	Math, logic, multi-step	Longer responses
System prompt	Setting consistent persona	Doesn't fix a single bad prompt

03 Writing Effective Prompts

Specificity and context turn vague answers into useful ones.

Most disappointing AI responses come down to one root cause: a vague prompt. The model has to guess what you actually want — and it often guesses wrong. Here's how to fix that.

Be specific

Vague prompt	Specific prompt
"Write something about dogs"	"Write a 100-word paragraph for a children's book about why dogs wag their tails"
"Help with my code"	"This Python function throws a KeyError on line 12 — here's the code and the error. Fix it and explain why it happened"
"Summarize this"	"Summarize this article in 3 bullet points, focused on the financial impact, for a non-expert audience"

Provide context

Tell the model who you are, what you need, and what format you want the answer in. The model can't read your mind — but it can follow clear instructions.

Context example

Weak:

"Explain photosynthesis"

Strong:

"I'm a 10th-grade biology teacher preparing a 5-minute explanation for students who have never studied chemistry. Explain photosynthesis using a simple analogy, avoid chemical formulas, and end with one fun fact."

Iterate — refine your prompt

Prompting is rarely 'one and done'. If the first answer is off-target, don't start over — refine. Tell the model what was wrong and what to change.

Iteration example

Turn 1: "Write a product description for a water bottle"

-> too generic, too long

Turn 2: "Make it shorter – 2 sentences max – and focus on the leak-proof lid and the fact it keeps drinks cold for 24 hours"

-> much closer

Turn 3: "Good, now make the tone more playful for a Gen-Z audience"

-> final version

Common mistakes to avoid

Too vague	"Make this better" — better how? Faster? Shorter? More formal? Specify the dimension.
Too long / cluttered	Stuffing 10 unrelated requests into one prompt confuses the model about priorities.
No context	Asking technical questions without saying your skill level leads to answers that are too basic or too advanced.
No output format	Not specifying format (list, table, code, JSON) means you'll get whatever the model defaults to.
Assuming memory	Each new conversation starts fresh — the model doesn't remember earlier sessions unless you restate context.

04 Using the Anthropic / OpenAI API

From chat window to Python script — calling an LLM programmatically.

Both Anthropic (Claude) and OpenAI (GPT) expose their models through a simple HTTP API. You send a request containing your prompt and some parameters, and get back the model's generated text.

Step 1 — Get an API key

Sign up on the provider's developer console (console.anthropic.com or platform.openai.com), create an API key, and store it securely — never hard-code it directly in shared code or commit it to GitHub.

Setup

```
# Store your key as an environment variable, NOT in your code
# In your terminal:
export ANTHROPIC_API_KEY='your-key-here'
# In Python, read it from the environment
import os
api_key = os.environ['ANTHROPIC_API_KEY']
```

Step 2 — Make your first API call

Python — Anthropic

```
# pip install anthropic
import anthropic

client = anthropic.Anthropic() # reads ANTHROPIC_API_KEY automatically
response = client.messages.create(
    model='claude-sonnet-4-6',
    max_tokens=300,
    messages=[
        {'role': 'user', 'content': 'Explain gravity in one sentence.'}
    ]
)
print(response.content[0].text)
```

Key parameters

Parameter	What it controls	Typical range
model	Which model version to use	e.g. claude-sonnet-4-6

max_tokens	Maximum length of the response	100 - 4000
temperature	Randomness / creativity of output	0.0 (focused) - 1.0 (creative)
messages	Conversation history (list of turns)	list of role/content dicts

Understanding temperature

Python

```
# temperature = 0 -> deterministic, focused, repeatable
# Good for: factual Q&A, code generation, data extraction
response = client.messages.create(
    model='claude-sonnet-4-6', max_tokens=100, temperature=0,
    messages=[{'role':'user','content':'What is 12 * 8?'}]
)

# temperature = 0.9 -> varied, creative, less predictable
# Good for: brainstorming, creative writing, varied suggestions
response = client.messages.create(
    model='claude-sonnet-4-6', max_tokens=100, temperature=0.9,
    messages=[{'role':'user','content':'Give me a creative name for a
    coffee shop'}]
)
```

Step 3 — A simple Q&A; script

Python

```
import anthropic

client = anthropic.Anthropic()

def ask(question, system_prompt=None):
    kwargs = dict(
        model='claude-sonnet-4-6',
        max_tokens=500,
        messages=[{'role': 'user', 'content': question}]
    )
    if system_prompt:
        kwargs['system'] = system_prompt
    response = client.messages.create(**kwargs)
    return response.content[0].text

# Try it
answer = ask(
    'Why is the sky blue?',
    system_prompt='You are a friendly science teacher for 8-year-olds.'
)
print(answer)
```

Cost awareness

API usage is billed per token (roughly, per word fragment) for both input and output. Set sensible `max_tokens` limits and avoid sending huge unnecessary context with every call to keep costs predictable.

05 Lab — Build a Study-Buddy Chatbot

A command-line chatbot that answers questions on any topic you give it.

In this lab, you'll combine a system prompt, conversation history, and a loop to build an interactive chatbot that stays focused on a topic you choose.

Step 1 — Set up the conversation loop

Python — setup

```
import anthropic
client = anthropic.Anthropic()
topic = input('What topic do you want to study today? ')
system_prompt = (
    f'You are a friendly, patient study buddy helping a student learn '
    f'about '
    f'{topic}. Explain concepts simply, use examples, and ask a follow-up '
    f'question after each answer to check understanding. Keep answers '
    f'under '
    f'150 words.'
)
conversation = [] # stores the full back-and-forth history
```

Step 2 — The chat loop

Python — chat loop

```
print(f"Study buddy ready! Ask me anything about {topic}.")
print("Type 'quit' to exit.\n")

while True:
    user_input = input('You: ')
    if user_input.lower() == 'quit':
        print('Good luck studying!')
        break

    # Add the user's message to history
    conversation.append({'role': 'user', 'content': user_input})

    # Call the API with full history so the model remembers context
    response = client.messages.create(
        model='claude-sonnet-4-6',
        max_tokens=400,
        system=system_prompt,
        messages=conversation
    )
    reply = response.content[0].text
    print(f'Buddy: {reply}\n')

    # Add the assistant's reply to history too – this is what gives
    # the chatbot 'memory' within this session
    conversation.append({'role': 'assistant', 'content': reply})
```

Step 3 — Try these extensions

Add a quiz mode	After 3 exchanges, have the bot generate a 5-question quiz on what was discussed.
Limit scope	In the system prompt, instruct the bot to politely redirect if asked about unrelated topics.
Save transcripts	Write the conversation list to a JSON file at the end of the session for later review.
Difficulty levels	Ask the user 'beginner / intermediate / advanced' at the start and adjust the system prompt accordingly.

Lab checkpoint

Your chatbot should remember earlier questions within the same session (try asking 'can you give an example of what you just explained?'). If it doesn't, check that you're appending BOTH user and assistant messages to the conversation list.

06 Quick Reference Cheat Sheet

Everything from today on one page.

Prompting techniques	
Zero-shot	Ask directly: 'Classify this review: ...'
Few-shot	Give 2-3 input/output examples first
CoT	Add: 'Think step by step before answering'
System prompt	Set role/persona separately from the question

Effective prompts	
Be specific	State exact format, length, audience
Add context	Who you are, what you need, why
Iterate	Refine based on what's wrong, don't restart
Avoid	Vague asks, no format, no context

API basics	
Install	<code>pip install anthropic</code>
Key call	<code>client.messages.create(model=..., max_tokens=..., messages=[...])</code>
temperature=0	Focused, deterministic output
temperature=0.9	Creative, varied output

Day 19 complete — LLMs & Prompt Engineering